МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интелекта
02.03.01 Математика и компьютерные науки

Отчет по лабораторной работе №1

Дискретная математика

«Операции над мультимножествами и код Грея»

Выполнил:
студент группы 5130201/50301 _____ Ташлыкова Е.В.
Принял:
Преподаватель _____ Востров А.В.

«____» _____

2026г.

Санкт-Петербург, 2026

Содержание

Введение 4

1 Математическое описание 5

1.1 Множества и мультимножества 5

1.2 Операции над мультимножествами 6

1.3 Алгоритм генерации бинарного кода Грея 9

2 Особенности реализации 10

2.1 Структура программы 10

2.2 Структура данных: класс MultiSet 10

2.3 Генерация кода Грея 12

2.4 Ввод и вывод 13

2.5 Операции над мультимножествами 16

2.6 Главная функция и меню 18

2.7 Защита от некорректного ввода 19

2.8 Оценка сложности 19

3 Результаты работы 20

Корректный ввод 20

3.1 Главное меню 20

3.2 Создание универсума и автоматическое заполнение . . . 20

3.3 Ручное заполнение 23

3.4 Выполнение операций 25

3.5 Граничные случаи 27

Некорректный ввод 29

3.6 Некорректный пункт меню 29

3.7 Некорректная разрядность 29

3.8 Мощность больше мощности универсума и неверный способ заполнения 30

3.9 Замеры времени 31
Заключение 33
Список литературы 36
Приложение А. Исходный код программы 38

Введение

Цель работы – получение практических навыков работы с мультимножествами, изучение алгоритма генерации бинарного кода Грея, а также реализация данных структур данных и алгоритмов в виде модульной программы на языке программирования C++.

Задание: разработать программу генерации бинарного кода Грея для формирования элементов универсума мультимножеств заданной пользователем разрядности $n \in [0, 20]$. На основе созданного универсума формируются два мультимножества A и B (вручную или автоматически по выбору пользователя) с заданными мощностями. Программа должна вычислять теоретико-множественные операции (объединение, пересечение, обе разности, симметрическую разность, дополнения до универсума) и арифметические операции (сумму, разности, произведение и деление). Также необходимо реализовать защиту от некорректного пользовательского ввода и интерактивное консольное меню.

Для достижения цели необходимо решить следующие задачи:

1. изучить определения мультимножества, универсума и операций над мультимножествами;
2. изучить алгоритм построения бинарного кода Грея и обосновать его корректность;
3. спроектировать структуру данных для хранения мультимножеств и разделить программу на модули;
4. реализовать генерацию универсума, ручное и автоматическое заполнение мультимножеств, все операции задания и консольное меню;
5. обеспечить защиту от некорректного ввода и протестировать программу, включая граничные случаи.

1 Математическое описание

1.1 Множества и мультимножества

Понятие множества принадлежит к числу фундаментальных понятий математики. Множество – это любая определенная совокупность объектов. Объекты, из которых составлено множество, называются его элементами, при этом все элементы множества различны. (Новиков, стр 23).

Если объект x является элементом множества M, то говорят, что x принадлежит M ($x \in M$). В противном случае говорят, что x не принадлежит M ($x \notin M$). Множество, не содержащее элементов, называется пустым и обозначается как $\varnothing$.

Мультимножеством называется множество, в котором допускается наличие одинаковых элементов. Пусть дано множество $X = x_1, \ldots, x_n$, тогда мулитмножеством над множеством X будет являться $\widehat{X} = [x_1^{a_1}, \ldots, x_n^{a_n}] =$

$= \langle x_1, \ldots, x_1; \ldots; x_n, \ldots, x_n \rangle = \langle a_1(x_1), \ldots, a_n(x_n) \rangle$. Это совокупность элементов множества X, в которую элемент x_i входит $a_i \geqslant 0$ раз.

Число a_i называется *показателем* элемента x_i, множество X – *носителем*, число $m = a_1 + \cdots + a_n$ – *мощностью*, а $\underline{\widehat{X}} = \{x_i \in X \mid a_i > 0\}$ называется *составом* мультимножества $\widehat{X}$.

Элемент x принадлежит мультимножеству $\widehat{X}$ ($x \in \widehat{X}$) и в мультимножестве $\widehat{X}$ имеется ровно k экземпляров элемента x тогда и только тогда, когда кратность элемента x равна $k_X(x) = k > 0$. Когда кратность элемента x равна нулю $k_X(x) = 0$, тогда элемент x не содержится в мультимножестве $\widehat{X}$ ($x \notin \widehat{X}$).

Принадлежность элемента x мультимножеству $\widehat{X}$ определяется функ-

цией кратности:

$$k_X(x) = \begin{cases} k > 0 \text{ – целое, если } x \in X, \\ 0, \text{ если } x \notin X. \end{cases}$$

В работе универсумом U называется мультимножество над множеством X кодов Грея, содержащее все допустимые элементы с максимальными кратностями. Мультимножество A является подмультимножеством универсума ($A \subseteq U$), если $m_A(x) \leqslant m_U(x)$ для всех $x \in X$.

Элементами множества X являются коды Грея разрядности n, а их кратности в U устанавливаются случайным образом из диапазона [0, 7].

1.2 Операции над мультимножествами

- Объединением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые присутствуют хотя бы в одном из мультимножеств, и кратность каждого элемента равна максимальной кратности соответствующих элементов в объединяемых мультимножествах

$$A \cup B = \{\, k_{A \cup B}(x) \mid k_{A \cup B}(x) = \max\{k_A(x),\, k_B(x)\} \,\}$$

- Пересечением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна минимальной кратности соответствующих элементов в пересекаемых мультимножествах

$$A \cap B = \{\, k_{A \cap B}(x) \mid k_{A \cap B}(x) = \min\{k_A(x),\, k_B(x)\} \,\}$$

- Дополнением мультимножества A до универсума U называется мультимножество, состоящее из всех элементов универсума и кратность каждого элемента равна разности кратностей соответствующих элементов в универсуме U и дополняемом мультимножестве A

$$\bar{A} = \{\, k_{\bar{A}}(x) \mid k_{\bar{A}}(x) = k_U(x) - k_A(x) \,\}$$

- Разностью (теоретико-множественной) называется мультимножество, состоящее из минимума кратностей первого мультимножества и разности кратностей универсума и второго мультимножества

$$A \setminus B = \{\, k_{A\setminus B}(x) \mid k_{A\setminus B}(x) = \min\{k_A(x),\, k_U(x) - k_B(x)\} \,\}$$

- Симметрической разностью мультимножеств A и B называется мультимножество, состоящее из всех элементов обоих мультимножеств, кратности которых равняются объединению кратностей двух разностей мультимножеств – из первого второго и из второго первого

$$A \triangle B = \{\, k_{A\triangle B}(x) \mid k_{A\triangle B}(x) =$$

$$= \max\{\min\{k_A(x),\, k_U(x) - k_B(x)\},\, \min\{k_B(x),\, k_U(x) - k_A(x)\}\} \,\} =$$

$$= (A \cup B) \setminus (A \cap B)$$

- Арифметической суммой мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые присутствуют хотя бы в одном из мультимножеств, и кратность каждого элемента равна сумме кратностей соответствующих элементов в складываемых муль-

тимножествах

$$A + B = \{\, k_{A+B}(x) \mid k_{A+B}(x) = \min\{k_A(x) + k_B(x),\, k_U(x)\} \,\}$$

- Арифметической разностью мультимножеств A и B называется мультимножество, состоящее из всех элементов мультимножества A, кратность каждого элемента равна разности кратностей соответствующих элементов в вычитаемых мультимножествах

$$A - B = \{\, k_{A-B}(x) \mid k_{A-B}(x) = \max\{k_A(x) - k_B(x),\, 0\} \,\}$$

- Арифметическим произведением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна произведению кратностей соответствующих элементов в перемножаемых мультимножествах

$$A * B = \{\, k_{A*B}(x) \mid k_{A*B}(x) = \min\{k_A(x) * k_B(x),\, k_U(x)\} \,\}$$

- Арифметическим делением мультимножеств A и B называется мультимножество, состоящее из всех элементов, которые одновременно присутствуют в каждом из мультимножеств, и кратность каждого элемента равна целочисленному делению кратностей соответствующих элементов, при условии, что кратность элемента в делителе не равна нулю

$$A : B = \begin{cases} \{\, k_{A:B}(x) \mid k_{A:B}(x) = [k_A(x) : k_B(x)] \,\}, & k_B(x) \neq 0 \\ 0, & k_B(x) = 0 \end{cases}$$

1.3 Алгоритм генерации бинарного кода Грея

n-Мерный код Грея – это последовательность всех 2^n бинарных слов длины n такая, что два соседних слова отличаются ровно в одном символе [2]. Известно несколько алгоритмов построения кодов Грея. Традиционно используют нерекурсивный алгоритм, основанный на операции побитового исключающего ИЛИ $\oplus$ исходного двоичного кода N-разрядного числа и его копии сдвинутой на один разряд вправо.

i-й код Грея вычисляется непосредственно по формуле

$$g(i) = i \oplus (i \gg 1), \qquad i = 0, 1, \ldots, 2^n - 1, \tag{1}$$

где $\oplus$ — побитовое исключающее «ИЛИ», а $\gg$ — сдвиг вправо на один разряд. Полученное число записывается в виде строки из n двоичных цифр с сохранением ведущих нулей. Далее, в таблице 1, приведен пример для $n = 3$.

Таблица 1: Код Грея для $n = 3$

i	Двоичный код i	$i \gg 1$	Код Грея $g(i)$
0	000	000	000
1	001	000	001
2	010	001	011
3	011	001	010
4	100	010	110
5	101	010	111
6	110	011	101
7	111	011	100

Каждый код формируется за $O(n)$ операций, поэтому построение всей последовательности имеет сложность $O(n \cdot 2^n)$. При $n = 0$ последовательность считается пустой, а универсум — пустым мультимножеством.

2 Особенности реализации

2.1 Структура программы

Программа написана на языке C++ (стандарт C++20) с использованием стандартной библиотеки [3] и состоит из одного класса и функции mainб которые являются реализацией консольного приложения для работы с мультимножествами.

2.2 Структура данных: класс MultiSet

Мультимножество хранится в классе `MultiSet` в виде ассоциативного контейнера `std::map<std::string, std::size_t>`: ключом является код Грея (строка), значением — его кратность. Контейнер хранит только элементы с положительной кратностью; метод `add` игнорирует добавление с нулевым количеством, поэтому инвариант «в `data_` нет нулевых кратностей» выполняется всегда. Это даёт разреженное представление (пустое мультимножество не занимает памяти под элементы универсума) и упорядоченный перебор элементов, благодаря чему вывод детерминирован. Объявление класса приведено в листинге 1.

Листинг 1: Объявление класса MultiSet (multiset.hpp)

```
#pragma once

#include <cstddef>
#include <map>
#include <string>

class MultiSet
{
public:
    using Count = std::size_t;
    using Data = std::map<std::string, Count>;
    MultiSet() = default;
    void clear();
    void add(const std::string &value, Count count = 1);
    Count count(const std::string &value) const;
    Count size() const;
    bool empty() const;
    const Data &data() const;
private:
    Data data_;
};
```

Методы класса и их сложность (k — число различных элементов в мультимножестве) перечислены в таблице 2.

Таблица 2: Методы класса MultiSet

Метод	Описание	Сложность
`clear()`	удаляет все элементы	$O(k)$
`add(value, count)`	увеличивает кратность элемента на `count`; при `count = 0` ничего не делает (по умолчанию добавляется одно вхождение)	$O(\log k)$
`count(value)`	возвращает кратность элемента, либо 0, если элемента нет; использует `find`, поэтому не создаёт записей в контейнере	$O(\log k)$
`size()`	мощность мультимножества — сумма всех кратностей	$O(k)$
`empty()`	признак пустоты (нет ни одного элемента с кратностью > 0)	$O(1)$
`data()`	константная ссылка на внутренний контейнер для перебора элементов	$O(1)$

2.3 Генерация кода Грея

Модуль `gray_code` содержит две функции. Функция `generateGrayCode(bits)` возвращает вектор из 2^{bits} строк: для каждого i вычисляется число $i \oplus (i \gg 1)$, которое переводится в строку функцией `decimalToBinary(value, width)`. Последняя строит строку ровно из `width` символов, извлекая биты от старшего к младшему выражением `(value >> shift) & 1`; поэтому ведущие нули сохраняются. При `bits = 0` возвращается пустой вектор. Реализация приведена в листинге 2.

Листинг 2: Функции генерации кода Грея (gray_code.cpp)

```
std::vector<std::string> generateGrayCode(std::size_t bits)
{
    std::vector<std::string> result;
    if (bits == 0) return result;
    const std::size_t count = static_cast<std::size_t>(1) << bits;
    result.reserve(count);
    for (std::size_t i = 0; i < count; ++i)
    {
        const unsigned long long gray = i ^ (i >> 1);
        result.push_back(decimalToBinary(gray, bits));
    }
    return result;
}

std::string decimalToBinary(unsigned long long value, std::size_t width)
{
    std::string result(width, '0');
    for (std::size_t i = 0; i < width; ++i)
    {
        const std::size_t shift = width - 1 - i;
        result[i] = ((value >> shift) & 1ULL) ? '1' : '0';
    }
    return result;
}
```

2.4 Ввод и вывод

Защищённое чтение числа. Функция `readInt(prompt, minValue, maxValue)` читает строку целиком функцией `std::getline`, помещает её в `std::stringstream` и принимает значение только в том случае, если одновременно выполняются три условия: число успешно извлечено, после него в строке нет посторонних символов и значение лежит в допустимом диапазоне. Такая схема исключает частичное чтение, свойственное `cin » x` (например, строка `12abc` была бы прочитана как 12). Число читается в `long long`, поэтому переполнение `int` не приводит к ошибке (значение просто не попадает в диапазон). При

некорректном вводе выводится сообщение с указанием диапазона и запрос повторяется (листинг 3).

Листинг 3: Функция защищённого ввода readInt (input.cpp)

```
int readInt(const std::string &prompt, int minValue, int maxValue)
{
    while (true)
    {
        std::cout << prompt;
        std::string line;
        std::getline(std::cin, line);
        std::stringstream ss(line);
        long long value;
        char extra;
        if ((ss >> value) && !(ss >> extra) && value >= minValue && value
   <= maxValue)
            return static_cast<int>(value);
        std::cout << "Ошибка: введите целое число от " << minValue << " до
    " << maxValue << ".\n";
    }
}
```

Ручное заполнение. Функция `readManualMultiset` показывает пользователю пронумерованный универсум, а затем для каждого из `power` вхождений запрашивает *номер* элемента (а не его код), что исключает ошибки в записи кодов. Введённый номер проверяется `readInt` на принадлежность диапазону $[1; 2^n]$; затем проверяется, что кратность элемента в формируемом мультимножестве ещё меньше его кратности в универсуме. Если лимит исчерпан, выводится сообщение, счётчик не увеличивается и ввод повторяется. Таким образом, условие $A \subseteq U$ выполняется по построению.

Автоматическое заполнение. Функция `generateAutomaticMultiset` реализует случайную выборку без возвращения («алгоритм мешка»):

1. формируется вектор `pool`, в который каждый элемент универсума по-

мещается столько раз, какова его кратность, — всего $|U|$ строк;

2. вектор перемешивается алгоритмом `std::shuffle` с генератором `std::mt19937_6` инициализируемым значением `std::random_device`;

3. первые `power` элементов добавляются в результат.

Элемент не может быть извлечён больше раз, чем он находится в мешке, поэтому условие $A \subseteq U$ выполняется без дополнительных проверок, а мощность результата равна `power`. Время и память — $O(|U|)$ (листинг 4).

Листинг 4: Автоматическое заполнение мультимножества (input.cpp)

```
MultiSet generateAutomaticMultiset(const MultiSet &universe,
                                   std::size_t power)
{
    MultiSet result;
    if (power == 0 || universe.empty()) return result;
    std::vector<std::string> pool;
    pool.reserve(universe.size());
    for (const auto &[value, count] : universe.data())
    {
        for (std::size_t i = 0; i < count; ++i)
            pool.push_back(value);
    }
    std::random_device rd;
    std::mt19937_64 generator(rd());
    const std::size_t actualPower = std::min(power, pool.size());
    std::shuffle(pool.begin(), pool.end(), generator);
    for (std::size_t i = 0; i < actualPower; ++i)
        result.add(pool[i]);
    return result;
}
```

Вывод. Функция `printUniverse` печатает универсум в порядке кода Грея с нумерацией, кратностями и общей мощностью; `printMultiset` печатает мультимножество в формате `A = { код (кратность), ... } |A| = мощность`, а пустое — как $\varnothing$.

2.5 Операции над мультимножествами

Модуль `operations` содержит по одной функции на каждую операцию задания; все функции возвращают новый объект `MultiSet`, не изменяя операнды. Соответствие функций формулам таблицы **??** показано в таблице 3. Для перебора достаточно одного прохода по элементам первого операнда (или обоих — при объединении), а кратность во втором операнде находится методом `count`, поэтому сложность каждой базовой операции составляет $O(k \log k)$.

Таблица 3: Функции модуля operations

Функция	Операция	Принцип реализации
`setUnion(a, b)`	$A \cup B$	копия A; для элементов B кратность увеличивается до максимума
`intersection(a, b)`	$A \cap B$	минимум кратностей по элементам A
`complement(a, u)`	$\overline{A}$	$m_U(x) - m_A(x)$ по всем элементам универсума
`difference(a, b, u)`	$A \setminus B$	`intersection(a, complement(b, u))`
`symmetricDifference(a, b, u)`	$A \triangle B$	`difference(setUnion(a, b), intersection(a, b), u)`
`arithmeticSum(a, b, u)`	$A + B$	сумма кратностей, ограниченная $m_U(x)$
`arithmeticDifference(a, b)`	$A - B$	разность кратностей, только положительные результаты
`arithmeticProduct(a, b, u)`	$A \cdot B$	произведение кратностей (при $m_B(x) > 0$), ограниченное $m_U(x)$
`arithmeticDivision(a, b)`	A / B	целая часть частного при $m_B(x) > 0$; при $m_B(x) = 0$ элемент не включается

Разность и симметрическая разность выражены композицией базовых операций, поэтому код дословно повторяет математические определения (листинг 5). Деление на нулевую кратность исключено явной проверкой, что предотвращает аварийное завершение.

Листинг 5: Фрагмент модуля операций (operations.cpp)

```
MultiSet complement(const MultiSet &a, const MultiSet &universe)
{
    MultiSet result;
    for (const auto &[value, countU] : universe.data())
    {
        const auto countA = a.count(value);
        result.add(value, countU - countA);
    }
    return result;
}

MultiSet difference(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe)
{
    return intersection(a, complement(b, universe));
}

MultiSet symmetricDifference(const MultiSet &a, const MultiSet &b, const
    MultiSet &universe)
{
    return difference(setUnion(a, b), intersection(a, b), universe);
}

MultiSet arithmeticDivision(const MultiSet &a, const MultiSet &b)
{
    MultiSet result;
    for (const auto &[value, countA] : a.data())
    {
        const auto countB = b.count(value);
        if (countB > 0 && countA >= countB)
            result.add(value, countA / countB);
    }
    return result;
}
```

2.6 Главная функция и меню

Состояние программы хранится в `main`: универсум `universe`, вектор `universeOrder` с кодами в порядке Грея (он задаёт нумерацию при ручном вводе) и мультимножества `a` и `b`. Максимальная разрядность ограничена константой `MAX_BITS = 20`, что соответствует $2^{20} = 1\,048\,576$ различным кодам. Меню содержит четыре пункта (таблица 4).

Таблица 4: Пункты меню

Пункт	Название	Действие
1	Создать новые мультимножества	запрос разрядности, генерация универсума, ввод мощностей и способов заполнения, создание A и B
2	Показать универсум и мультимножества	вывод U, A и B
3	Выполнить операции над мультимножествами	вывод A, B и результатов всех операций
0	Выход	завершение программы

Функция `createMultisets` выполняет пункт 1 в следующем порядке:

1. генерирует коды Грея заданной разрядности и очищает предыдущие данные;
2. при разрядности 0 сообщает, что универсум, A и B пусты, и завершает работу (граничный случай);
3. назначает каждому коду случайную кратность из $[1; 7]$ (равномерное распределение `std::uniform_int_distribution`, генератор `std::mt19937_64`);
4. выводит универсум и время выполнения этапов (генерация кодов, заполнение, печать), измеренное с помощью `std::chrono::steady_clock`;
5. запрашивает мощности A и B в диапазоне $[0; |U|]$ и способы их заполнения (1 — вручную, 2 — автоматически);
6. заполняет мультимножества выбранным способом и выводит их.

Функция `printOperations` печатает результаты операций: $A \cup B$, $A \cap B$, $A \setminus B$, $B \setminus A$, $A \triangle B$, $\overline{A}$, $\overline{B}$, а также $A+B$, $A-B$, $B-A$, $A \cdot B$, A/B и B/A. Вычитание, деление не коммутативны, поэтому выводятся оба варианта; произведение и сумма коммутативны и выводятся один раз.

2.7 Защита от некорректного ввода

Реакция программы на некорректные действия пользователя приведена в таблице 5.

Таблица 5: Реакция программы на некорректный ввод

Некорректный ввод	Реакция программы
нечисловая строка (`abc`), число с посторонними символами (`12abc`), дробное число (`1.5`), два числа (`3 7`), пустая строка	сообщение «Ошибка: введите целое число от … до …», повторный запрос
число вне диапазона (разрядность > 20 или < 0, мощность $> \lvert U \rvert$, пункт меню, отличный от 0–3)	то же сообщение с указанием допустимого диапазона
число, не помещающееся в `long long`	значение отклоняется как нечисловое, программа не завершается аварийно
пробелы по краям числа (`  5  `)	значение принимается как 5
номер элемента вне $[1; 2^n]$ при ручном вводе	сообщение об ошибке, повторный запрос
элемент уже использован максимальное число раз при ручном вводе	сообщение с указанием максимальной кратности, ввод повторяется, счётчик вхождений не увеличивается
способ заполнения, отличный от 1 и 2	сообщение об ошибке, повторный запрос
разрядность 0	пустые универсум, A и B; операции дают пустые результаты
пункты 2 и 3 до создания мультимножеств	выводятся пустые универсум и мультимножества, аварийного завершения нет

2.8 Оценка сложности

Генерация универсума имеет сложность $O(n \cdot 2^n)$ по времени, заполнение контейнера — $O(2^n \cdot n)$ (каждая вставка в `std::map` сравнивает строки

длины n), вывод — $O(2^n)$. Автоматическое заполнение требует $O(|U|)$ времени и памяти, ручное — $O(\texttt{power} \cdot \log 2^n)$. Операции над мультимножествами выполняются за $O(k \log k)$, где $k \leq 2^n$ — число различных элементов. Экспериментальные замеры приведены в п. 3.9.

3 Результаты работы

Ниже приведены протоколы работы программы, полученные при запуске собранного приложения (сборка: `g++ -std=c++17 -O1 *.cpp`). Значения кратностей универсума и результаты автоматического заполнения случайны и от запуска к запуску различаются. Введённые пользователем данные в протоколах показаны после приглашений.

Корректный ввод

3.1 Главное меню

После запуска программа выводит меню из четырёх пунктов и ожидает выбора (рисунок 1).

```
1. Создать новые мультимножества
2. Показать универсум и мультимножества
3. Выполнить операции над мультимножествами
0. Выход
Выберите пункт:
```

Рис. 1: Главное меню

3.2 Создание универсума и автоматическое заполнение

Выбирается пункт 1, вводится разрядность 3 (допустимый диапазон — от 0 до 20). Программа формирует 8 кодов Грея в порядке кода Грея со случайными кратностями из $[1; 7]$ и выводит универсум с его мощностью. Затем

задаются мощности $A = 5$ и $B = 4$ и автоматический способ заполнения (рисунок 2). Пункт 2 повторно выводит универсум и оба мультимножества.

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 3

Универсум (8 уникальных элементов, общая мощность 35):
1: 000 (кратность 6)
2: 001 (кратность 2)
3: 011 (кратность 7)
4: 010 (кратность 3)
5: 110 (кратность 6)
6: 111 (кратность 3)
7: 101 (кратность 4)
8: 100 (кратность 4)

Введите мощность мультимножества A: 5
Введите мощность мультимножества B: 4

Способ заполнения A (1 - вручную, 2 - автоматически): 2
Способ заполнения B (1 - вручную, 2 - автоматически): 2

СОЗДАННЫЕ МНОЖЕСТВА:
A = { 000 (2), 011 (1), 100 (1), 110 (1) }  |A| = 5
B = { 000 (1), 010 (1), 101 (1), 111 (1) }  |B| = 4

Выберите пункт: 2

Универсум (8 уникальных элементов, общая мощность 35):
1: 000 (кратность 6)
2: 001 (кратность 2)
3: 011 (кратность 7)
4: 010 (кратность 3)
5: 110 (кратность 6)
6: 111 (кратность 3)
7: 101 (кратность 4)
8: 100 (кратность 4)
A = { 000 (2), 011 (1), 100 (1), 110 (1) }  |A| = 5
B = { 000 (1), 010 (1), 101 (1), 111 (1) }  |B| = 4

Выберите пункт: 0
```

Рис. 2: Создание универсума и автоматическое заполнение A и B

Мощности полученных мультимножеств равны заданным ($|A| = 5, |B| = 4$), кратности не превышают кратностей универсума.

3.3 Ручное заполнение

Выбирается ручной способ заполнения для A (мощность 8) и автоматический для B (мощность 0). В протоколе (рисунок 3) намеренно вводятся ошибочные значения: номер элемента вне диапазона, нечисловая строка, а также повторный ввод элемента, кратность которого в универсуме уже исчерпана.

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 3

Универсум (8 уникальных элементов, общая мощность 34):
1: 000 (кратность 3)
2: 001 (кратность 4)
3: 011 (кратность 7)
4: 010 (кратность 2)
5: 110 (кратность 2)
6: 111 (кратность 7)
7: 101 (кратность 6)
8: 100 (кратность 3)

Введите мощность мультимножества A: 8
Введите мощность мультимножества B: 0

Способ заполнения A (1 - вручную, 2 - автоматически): 1
Способ заполнения B (1 - вручную, 2 - автоматически): 2

Ручное заполнение мультимножества A.
Введите номер элемента универсума для каждого вхождения.
Элемент 1: 9
Ошибка: введите целое число от 1 до 8.
Элемент 1: abc
Ошибка: введите целое число от 1 до 8.
Элемент 1: 1
Элемент 2: 1
Элемент 3: 1
Элемент 4: 1
Ошибка: элемент 000 уже использован максимальное число раз (3).
Элемент 4: 2
Элемент 5: 2
Элемент 6: 2
Элемент 7: 2
Элемент 8: 2
Ошибка: элемент 001 уже использован максимальное число раз (4).
Элемент 8: 3

СОЗДАННЫЕ МНОЖЕСТВА:
A = { 000 (3), 001 (4), 011 (1) }  |A| = 8
B = ∅  |B| = 0
```



```
Выберите пункт: 0
```

Мультимножество A сформировано в точном соответствии с допустимыми введёнными данными: при превышении лимита (000 — 3 вхождения, 001 — 4) программа сообщила об ошибке и запросила другой элемент, а мощность $|A| = 8$ равна заданной.

3.4 Выполнение операций

Сначала выбирается пункт 1: вводится разрядность 3, мощности $A = 8$ и $B = 6$ и автоматический способ заполнения обоих мультимножеств (рисунок 4).

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 3

Универсум (8 уникальных элементов, общая мощность 47):
1: 000 (кратность 4)
2: 001 (кратность 7)
3: 011 (кратность 7)
4: 010 (кратность 7)
5: 110 (кратность 7)
6: 111 (кратность 7)
7: 101 (кратность 6)
8: 100 (кратность 2)

Введите мощность мультимножества A: 8
Введите мощность мультимножества B: 6

Способ заполнения A (1 - вручную, 2 - автоматически): 2
Способ заполнения B (1 - вручную, 2 - автоматически): 2

СОЗДАННЫЕ МНОЖЕСТВА:
A = { 001 (1), 010 (2), 011 (1), 101 (1), 110 (1), 111 (2) }  |A| = 8
B = { 010 (1), 011 (1), 101 (2), 110 (2) }  |B| = 6
```

Рис. 4: Универсум и мультимножества A и B для проверки операций

Затем выбирается пункт 3. Программа выводит результаты всех операций над A и B (рисунок 5).

```
Выберите пункт: 3
A = { 001 (1), 010 (2), 011 (1), 101 (1), 110 (1), 111 (2) }  |A| = 8
B = { 010 (1), 011 (1), 101 (2), 110 (2) }  |B| = 6

ОПЕРАЦИИ НАД МУЛЬТИМНОЖЕСТВАМИ:

A ∪ B = { 001 (1), 010 (2), 011 (1), 101 (2), 110 (2), 111 (2) }  |A ∪ B|
    = 10
A ∩ B = { 010 (1), 011 (1), 101 (1), 110 (1) }  |A ∩ B| = 4
A \ B = { 001 (1), 010 (2), 011 (1), 101 (1), 110 (1), 111 (2) }  |A \ B|
    = 8
B \ A = { 010 (1), 011 (1), 101 (2), 110 (2) }  |B \ A| = 6
A △ B = { 001 (1), 010 (2), 011 (1), 101 (2), 110 (2), 111 (2) }  |A △ B|
    = 10
Ā = { 000 (4), 001 (6), 010 (5), 011 (6), 100 (2), 101 (5), 110 (6), 111
    (5) }  |Ā| = 39
B̄ = { 000 (4), 001 (7), 010 (6), 011 (6), 100 (2), 101 (4), 110 (5), 111
    (7) }  |B̄| = 41

A + B = { 001 (1), 010 (3), 011 (2), 101 (3), 110 (3), 111 (2) }  |A + B|
    = 14
A - B = { 001 (1), 010 (1), 111 (2) }  |A - B| = 4
B - A = { 101 (1), 110 (1) }  |B - A| = 2
A * B = { 010 (2), 011 (1), 101 (2), 110 (2) }  |A * B| = 7
A / B = { 010 (2), 011 (1) }  |A / B| = 3
B / A = { 011 (1), 101 (2), 110 (2) }  |B / A| = 5
```

Рис. 5: Результаты операций над A и B

Для проверки правильности вычислим результаты вручную по формулам таблицы **??** для трёх элементов (таблица 6). Значения a, b, u взяты из протоколов на рисунках 4 и 5.

Таблица 6: Проверка результатов операций вручную

	$x = 010$	$x = 101$	$x = 111$
$(a;\, b;\, u)$	$(2;\, 1;\, 7)$	$(1;\, 2;\, 6)$	$(2;\, 0;\, 7)$
$A \cup B$	2	2	2
$A \cap B$	1	1	0
$A \setminus B$	$\min(2, 6) = 2$	$\min(1, 4) = 1$	$\min(2, 7) = 2$
$B \setminus A$	$\min(1, 5) = 1$	$\min(2, 5) = 2$	$\min(0, 5) = 0$
$A \triangle B$	$\min(2, 6) = 2$	$\min(2, 5) = 2$	$\min(2, 7) = 2$
$\overline{A}$	$7 - 2 = 5$	$6 - 1 = 5$	$7 - 2 = 5$
$\overline{B}$	$7 - 1 = 6$	$6 - 2 = 4$	$7 - 0 = 7$
$A + B$	$\min(3, 7) = 3$	$\min(3, 6) = 3$	2
$A - B$	1	0	2
$B - A$	0	1	0
$A \cdot B$	$\min(2, 7) = 2$	$\min(2, 6) = 2$	0
A/B	$\lfloor 2/1 \rfloor = 2$	$\lfloor 1/2 \rfloor = 0$	0
B/A	$\lfloor 1/2 \rfloor = 0$	$\lfloor 2/1 \rfloor = 2$	0

Значения совпадают с выводом программы: например, в рисунке 5 для элемента `010` присутствуют $A \cup B$ — 2, $A \cap B$ — 1, $A + B$ — 3, $A - B$ — 1, а для `101` — B/A равно 2 и A/B равно 0 (элемент в результат не входит).

3.5 Граничные случаи

При разрядности 0 универсум и оба мультимножества пусты; операции над пустыми мультимножествами также дают пустые результаты, программа не завершается аварийно (рисунок 6).

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 0

Разрядность равна 0.
U = ∅  |U| = 0
A = ∅  |A| = 0
B = ∅  |B| = 0

Выберите пункт: 3
A = ∅  |A| = 0
B = ∅  |B| = 0

ОПЕРАЦИИ НАД МУЛЬТИМНОЖЕСТВАМИ:

A ∪ B = ∅  |A ∪ B| = 0
A ∩ B = ∅  |A ∩ B| = 0
A \ B = ∅  |A \ B| = 0
B \ A = ∅  |B \ A| = 0
A △ B = ∅  |A △ B| = 0
Ā = ∅  |Ā| = 0
B̄ = ∅  |B̄| = 0

A + B = ∅  |A + B| = 0
A - B = ∅  |A - B| = 0
B - A = ∅  |B - A| = 0
A * B = ∅  |A * B| = 0
A / B = ∅  |A / B| = 0
B / A = ∅  |B / A| = 0

Выберите пункт: 0
```

Рис. 6: Разрядность 0: пустые мультимножества и результаты операций

Некорректный ввод

3.6 Некорректный пункт меню

В меню вводятся `abc`, `7`, `-1` и пустая строка. Каждый ввод отклоняется с сообщением о допустимом диапазоне, затем вводится `0` — выход (рисунок 7).

```
Выберите пункт: abc
Ошибка: введите целое число от 0 до 3.
Выберите пункт: 7
Ошибка: введите целое число от 0 до 3.
Выберите пункт: -1
Ошибка: введите целое число от 0 до 3.
Выберите пункт:
Ошибка: введите целое число от 0 до 3.
Выберите пункт: 0
```

Рис. 7: Некорректный ввод в меню

3.7 Некорректная разрядность

При создании универсума вводятся `21`, `-1`, `12abc` и `a12`. Все значения отклоняются, после чего принимается корректное значение 2 (рисунок 8).

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 21
Ошибка: введите целое число от 0 до 20.

Введите разрядность бинарного кода Грея: -1
Ошибка: введите целое число от 0 до 20.

Введите разрядность бинарного кода Грея: 12abc
Ошибка: введите целое число от 0 до 20.

Введите разрядность бинарного кода Грея: a12
Ошибка: введите целое число от 0 до 20.

Введите разрядность бинарного кода Грея: 2

Универсум (4 уникальных элементов, общая мощность 19):
1: 00 (кратность 5)
2: 01 (кратность 4)
3: 11 (кратность 5)
4: 10 (кратность 5)
```

Рис. 8: Некорректная разрядность кода Грея

3.8 Мощность больше мощности универсума и неверный способ заполнения

При вводе мощности 99 при $|U| = 24$ и нечисловой строки ввод отклоняется. Для способа заполнения введено недопустимое значение 7 (рисунок 9).

```
Выберите пункт: 1

Введите разрядность бинарного кода Грея: 2

Универсум (4 уникальных элементов, общая мощность 24):
1: 00 (кратность 6)
2: 01 (кратность 5)
3: 11 (кратность 6)
4: 10 (кратность 7)

Введите мощность мультимножества A: 99
Ошибка: введите целое число от 0 до 24.

Введите мощность мультимножества A: abc
Ошибка: введите целое число от 0 до 24.

Введите мощность мультимножества A: 2
Введите мощность мультимножества B: 3

Способ заполнения A (1 - вручную, 2 - автоматически): 7
Ошибка: введите целое число от 1 до 2.

Способ заполнения A (1 - вручную, 2 - автоматически): 2
Способ заполнения B (1 - вручную, 2 - автоматически): 2

СОЗДАННЫЕ МНОЖЕСТВА:
A = { 01 (1), 10 (1) }  |A| = 2
B = { 00 (1), 01 (1), 11 (1) }  |B| = 3

Выберите пункт: 0
```

Рис. 9: Мощность больше $|U|$ и неверный способ заполнения

3.9 Замеры времени

Для оценки масштабируемости программа выводит время генерации кодов, заполнения универсума и печати. В таблице 7 приведены результаты для разных разрядностей. Замеры ориентировочные: получены при сборке

`-O1`, вывод перенаправлен в канал, кратности универсума случайны.

Таблица 7: Время выполнения этапов создания универсума

n	2^n	$\lvert U\rvert$	Генерация кодов, мс	Заполнение, мс	Печать, мс
10	1 024	4 068	< 1	< 1	< 1
14	16 384	65 991	3	6	9
16	65 536	261 463	4	29	36
18	262 144	1 046 518	26	84	112
20	1 048 576	4 193 325	69	381	491

Время растёт примерно пропорционально 2^n, что согласуется с оценкой $O(n \cdot 2^n)$; даже при максимальной разрядности $n = 20$ создание универсума занимает менее секунды.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована программа на C++, которая:

- генерирует бинарный код Грея разрядности $n \in [0; 20]$ по формуле $g(i) = i \oplus (i \gg 1)$;
- формирует универсум из 2^n кодов Грея со случайными кратностями из диапазона $[1; 7]$;
- формирует мультимножества A и B заданной мощности вручную или автоматически;
- вычисляет и выводит объединение, пересечение, разность (оба варианта), симметрическую разность, дополнение (оба варианта), арифметические сумму, разность, произведение и деление;
- защищена от некорректного ввода и управляется через меню.

Достоинства реализации:

- чёткое разделение ответственности между модулями (хранение, генерация, операции, ввод-вывод, интерфейс);
- разреженное хранение мультимножества в `std::map`: в памяти находятся только элементы с положительной кратностью, вывод упорядочен и детерминирован;
- разность и симметрическая разность выражены через базовые операции, поэтому код совпадает с математическими определениями;
- условие $A \subseteq U$ гарантируется при обоих способах заполнения; автоматическое заполнение не требует дополнительных проверок;
- строка ввода читается и проверяется целиком, что исключает частичное чтение значений;

- встроенные замеры времени позволяют оценить производительность при больших разрядностях.

Недостатки реализации:

- автоматическое заполнение требует $O(|U|)$ дополнительной памяти на «мешок» из строк (при $n = 20$ — порядка нескольких миллионов строк);
- при больших разрядностях печать всего универсума (2^n строк) занимает большую часть времени и засоряет консоль;
- значения кратностей хранятся в `std::size_t`, а строки кодов дублируются в ключах `std::map`, что увеличивает расход памяти по сравнению с вектором кратностей, индексированным номером кода;
- не предусмотрена обработка завершения потока ввода (Ctrl+D / EOF);
- результаты операций нельзя сохранить в файл или использовать как операнды следующей операции.

Масштабирование. На основе реализованных функций программу можно дополнить с минимальными изменениями кода:

- пунктом меню, сохраняющим результат выбранной операции в A или B (присваиванием возвращённого функцией объекта), что позволит строить цепочки вычислений;
- проверкой отношений между мультимножествами: включением $A \subseteq B$ ($m_A(x) \leq m_B(x)$ для всех x) и равенством $A = B$ — одной функцией, возвращающей `bool`, за один проход по элементам;
- операциями над тремя и более мультимножествами, например $A \cap B \cap C$ или $(A \cup B) \setminus C$, так как функции операций возвращают новый объект и применяются последовательно;
- арифметической степенью A^k с кратностями $\min(m_A(x)^k,\ m_U(x))$ и

умножением на число $k \cdot A$ — повторным применением произведения и суммы;

- вводом и выводом мультимножеств в файл: достаточно направить `printMultiset` в `std::ostream` и реализовать чтение пар «код; кратность» с той же проверкой диапазонов, что и в `readInt`;
- заменой строкового ключа на целочисленный индекс кода (или битовое представление), что уменьшит расход памяти и ускорит операции.

СПИСОК ЛИТЕРАТУРЫ

Литература

[1] Новиков Ф. А. Дискретная математика для программистов: учебник для вузов. — 3-е изд. — СПб.: Питер, 2009. — 384 с.

[2] Knuth D. E. The Art of Computer Programming. Vol. 4A: Combinatorial Algorithms, Part 1. — Upper Saddle River: Addison-Wesley, 2011. — 883 p.

[3] cppreference.com: справочник по языку C++ и стандартной библиотеке [Электронный ресурс]. — URL: https://en.cppreference.com (дата обращения: 24.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

Листинг 6: main.cpp

```
#include "gray_code.hpp"
#include "input.hpp"
#include "operations.hpp"

#include <random>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

#include <chrono>

void printOperations(const MultiSet &a, const MultiSet &b, const MultiSet
    &universe)
{
    std::cout << "\nОПЕРАЦИИ НАД МУЛЬТИМНОЖЕСТВАМИ:\n\n";
    printMultiset("A ∪ B", setUnion(a, b));
    printMultiset("A ∩ B", intersection(a, b));
    printMultiset("A \\ B", difference(a, b, universe));
    printMultiset("B \\ A", difference(b, a, universe));
    printMultiset("A △ B", symmetricDifference(a, b, universe));
    printMultiset("$\bar{A}$", complement(a, universe));
    printMultiset("$\bar{B}$", complement(b, universe));
    std::cout << "\n";
    printMultiset("A + B", arithmeticSum(a, b, universe));
    printMultiset("A - B", arithmeticDifference(a, b));
    printMultiset("B - A", arithmeticDifference(b, a));
    printMultiset("A * B", arithmeticProduct(a, b, universe));
    printMultiset("A / B", arithmeticDivision(a, b));
    printMultiset("B / A", arithmeticDivision(b, a));
}

bool createMultisets(std::size_t &bits, MultiSet &universe, std::vector<
    std::string> &universeOrder, MultiSet &a, MultiSet &b)
{
    const auto t0 = std::chrono::steady_clock::now();
    std::vector<std::string> codes;
    codes = generateGrayCode(bits);
    const auto t1 = std::chrono::steady_clock::now();
    universe.clear();
    universeOrder.clear();
    if (bits == 0)
    {
```

Листинг 7: multiset.hpp

```
#pragma once

#include <cstddef>
#include <map>
#include <string>

class MultiSet
{
public:
    using Count = std::size_t;
    using Data = std::map<std::string, Count>;
    MultiSet() = default;
    void clear();
    void add(const std::string &value, Count count = 1);
    Count count(const std::string &value) const;
    Count size() const;
    bool empty() const;
    const Data &data() const;

private:
    Data data_;
};
```

Листинг 8: multiset.cpp

```
#include "multiset.hpp"

void MultiSet::clear()
{
    data_.clear();
}

void MultiSet::add(const std::string &value, Count count)
{
    if (count == 0)
        return;
    data_[value] += count;
}

MultiSet::Count MultiSet::count(const std::string &value) const
{
    const auto it = data_.find(value);
    return it == data_.end() ? 0 : it->second;
}

MultiSet::Count MultiSet::size() const
{
    Count result = 0;
    for (const auto &[_, count] : data_)
        result += count;
    return result;
}

bool MultiSet::empty() const
{
    return data_.empty();
}

const MultiSet::Data &MultiSet::data() const
{
    return data_;
}
```

Листинг 9: input.hpp

```
#pragma once

#include "multiset.hpp"

#include <cstddef>
#include <string>
#include <vector>

int readInt(const std::string &prompt, int minValue, int maxValue);
MultiSet readManualMultiset(const std::string &name, const MultiSet &
    universe,
                            const std::vector<std::string> &order, std::
    size_t power);

MultiSet generateAutomaticMultiset(const MultiSet &universe, std::size_t
    power);

void printUniverse(const MultiSet &universe, const std::vector<std::string
    > &order);

void printMultiset(const std::string &name, const MultiSet &set);
```

Листинг 10: input.cpp

```
#include "input.hpp"

#include <algorithm>
#include <iostream>
#include <limits>
#include <random>
#include <sstream>

int readInt(const std::string &prompt, int minValue, int maxValue)
{
    while (true)
    {
        std::cout << prompt;
        std::string line;
        std::getline(std::cin, line);
        std::stringstream ss(line);
        long long value;
        char extra;
        if ((ss >> value) && !(ss >> extra) && value >= minValue && value
    <= maxValue)
            return static_cast<int>(value);
        std::cout << "Ошибка: введите целое число от " << minValue << " до
     " << maxValue << ".\n";
    }
}

MultiSet readManualMultiset(const std::string &name, const MultiSet &
    universe,
                            const std::vector<std::string> &order, std::
    size_t power)
{
    MultiSet result;
    if (power == 0 || universe.empty())
        return result;
    std::cout << "\nРучное заполнение мультимножества " << name << ".\n";
    std::cout << "Введите номер элемента универсума для каждого вхождения
    .\n";
    for (std::size_t i = 0; i < power; ++i)
    {
        const std::size_t index = readInt(
            "Элемент " + std::to_string(i + 1) + ": ", 1,
            order.size());
        const std::string &chosen = order[index - 1];
```

Листинг 11: operations.hpp

```
#pragma once

#include "multiset.hpp"

#include <string>
#include <vector>

MultiSet setUnion(const MultiSet& a, const MultiSet& b);
MultiSet intersection(const MultiSet& a, const MultiSet& b);
MultiSet difference(const MultiSet& a, const MultiSet& b, const MultiSet&
    universe);
MultiSet symmetricDifference(const MultiSet& a, const MultiSet& b, const
    MultiSet& universe);

MultiSet complement(const MultiSet& a, const MultiSet& universe);

MultiSet arithmeticSum(const MultiSet& a, const MultiSet& b, const
    MultiSet& universe);
MultiSet arithmeticDifference(const MultiSet& a, const MultiSet& b);
MultiSet arithmeticProduct(const MultiSet& a, const MultiSet& b, const
    MultiSet& universe);
MultiSet arithmeticDivision(const MultiSet& a, const MultiSet& b);
```

Листинг 12: operations.cpp

```
#include "operations.hpp"

#include <algorithm>

MultiSet setUnion(const MultiSet &a, const MultiSet &b)
{
    MultiSet result = a;
    for (const auto &[value, countB] : b.data())
    {
        const auto countA = result.count(value);
        if (countB > countA)
            result.add(value, countB - countA);
    }
    return result;
}

MultiSet intersection(const MultiSet &a, const MultiSet &b)
{
    MultiSet result;
    for (const auto &[value, countA] : a.data())
    {
        const auto countB = b.count(value);
        const auto count = std::min(countA, countB);
        result.add(value, count);
    }
    return result;
}

MultiSet complement(const MultiSet &a, const MultiSet &universe)
{
    MultiSet result;
    for (const auto &[value, countU] : universe.data())
    {
        const auto countA = a.count(value);
        result.add(value, countU - countA);
    }
    return result;
}

MultiSet difference(const MultiSet &a, const MultiSet &b, const MultiSet &
    universe)
{
    return intersection(a, complement(b, universe));
```

Листинг 13: gray_code.hpp

```
#pragma once

#include <cstddef>
#include <string>
#include <vector>

std::vector<std::string> generateGrayCode(std::size_t bits);
std::string decimalToBinary(unsigned long long value, std::size_t width);
```

Листинг 14: gray_code.cpp

```cpp
#include "gray_code.hpp"

#include <limits>
#include <stdexcept>

std::vector<std::string> generateGrayCode(std::size_t bits)
{
    std::vector<std::string> result;
    if (bits == 0) return result;
    const std::size_t count = static_cast<std::size_t>(1) << bits;
    result.reserve(count);
    for (std::size_t i = 0; i < count; ++i)
    {
        const unsigned long long gray = i ^ (i >> 1);
        result.push_back(decimalToBinary(gray, bits));
    }
    return result;
}

std::string decimalToBinary(unsigned long long value, std::size_t width)
{
    std::string result(width, '0');
    for (std::size_t i = 0; i < width; ++i)
    {
        const std::size_t shift = width - 1 - i;
        result[i] = ((value >> shift) & 1ULL) ? '1' : '0';
    }
    return result;
}
```